

HireHub - Today's Learning Summary

Beginner-friendly Hinglish notes: Local Storage, Applied Jobs, and conditional UI

1. localStorage basics

- **localStorage** browser me **data store karta hai**, but har value string format me store hoti hai.
- Array ya object save karne se pehle **JSON.stringify()** use karte hain.
- Wapas data read karte time **JSON.parse()** string ko array/object me convert karta hai.

```
localStorage.setItem("savedJobs", JSON.stringify(currentJobs));  
JSON.parse(localStorage.getItem("savedJobs"))
```

2. Saved Jobs ko refresh ke baad bhi rakhna

- Save button click hone par updated array state me set hota hai aur same updated array localStorage me bhi save hota hai.
- App start/refresh hone par saved jobs localStorage se state me load hote hain.
- `|| []` fallback important hai. Agar localStorage me key nahi hai, **getItem()** null deta hai. Null par **.length** use karne se error aata hai: **null.length** invalid hai.

```
const [savedJobs, setSavedJobs] = useState(  
  JSON.parse(localStorage.getItem("savedJobs")) || []  
);
```

3. Delete karte time localStorage sync rakhna

- Delete ke liye **filter()** se naya array banaya jisme removed job nahi hoti.
- Uske baad **setSavedJobs(updatedJobs)** se React UI update hota hai.
- Aur **localStorage.setItem(...)** se browser storage bhi same updated list se update hoti hai.
- Rule: state aur localStorage dono ko same data se update karo.

```
const updatedJobs = savedJobs.filter((job) => job.id !== id);  
setSavedJobs(updatedJobs);  
localStorage.setItem("savedJobs", JSON.stringify(updatedJobs));
```

4. Applied Jobs feature

- Saved jobs se separate **appliedJobs** state banayi. Isme wahi jobs aati hain jin par user ne Apply Now click kiya.
- Applied jobs bhi localStorage key **"appliedJobs"** me persist hote hain, isliye refresh ke baad bhi dikhte hain.
- Duplicate application rokne ke liye **some()** use kiya. Ye true/false return karta hai: same id already present hai ya nahi.

```
if (!appliedJobs.some((appliedJob) => appliedJob.id === job.id)) {  
  const currentJobs = [...appliedJobs, job];  
  setAppliedJobs(currentJobs);  
  localStorage.setItem("appliedJobs", JSON.stringify(currentJobs));  
}
```

5. Function props ka data flow

- Apply Now button **JobCard** me hai, lekin appliedJobs state **App.jsx** me hai.
- Isliye function prop ka flow tha: **App -> Jobs / SavedJobs -> JobCard**.
- JobCard me click par **handleAppliedJobs(job)** call hota hai. Job object pass karna zaroori hai, taki App ko pata chale kis job ke liye apply kiya gaya.

6. Applied Jobs page and UI

- **AppliedJobs** ek page hai, component folder me nahi, because uska apna route hai: **/applied-jobs**.
- App.jsx route se **appliedJobs** prop page ko pass hota hai; page **map()** se cards render karta hai.
- Navbar me **Applied Jobs** Link add kiya, so user page par navigate kar sakta hai.
- **isApplied={true}** prop use karke Applied Jobs page me Apply Now aur Save Job buttons hide kiye. Ye conditional rendering ka real use case hai.

7. Next planned feature - Application Status Tracking

- Har original job ka status jobs.js me add nahi karna chahiye, because job data sab users ke liye common hai.
- Status user ki application se related hai. Jab user apply karega, tab applied-job object me status add hoga.
- Example: **{ ...job, status: "Applied" }**. Later status ko Interview, Selected, ya Rejected me change kar sakenge.

Quick memory line: localStorage string rakhta hai; JSON.stringify save karta hai; JSON.parse wapas data banata hai.